Advanced programming Assignment

ROLL NO: CSB24065

NAME: Gourisankar Bhuyan

Session: Spring **Year:2026**

Question:

Develop a student performance analyzer in Java. You are given a list of students of your batch. Each student has: id (int) //don't include CSB string name (String) courses (List<String>) scores (Map<String, Integer>) where key = course, value = marks

Do: 1. Store students using appropriate collections.

2. Implement the following methods:

List<Student>

getTopNStudents(List<Student> students, int n);

Map<String, Double>

getAverageScorePerCourse(List<Student> students);

Set<String>

getAllUniqueCourses(List<Student> students);

Must use:

1. Use ArrayList, HashMap, and HashSet

2. Use Streams for aggregation and filtering

3. Sort students by average score (descending)

4. Use Comparator

5. Handle missing course scores using getOrDefault

6. Ensure type safety using generics Perform

complexity analysis:

1. What is the time complexity of computing course averages?

2. What is the complexity of sorting top N students?

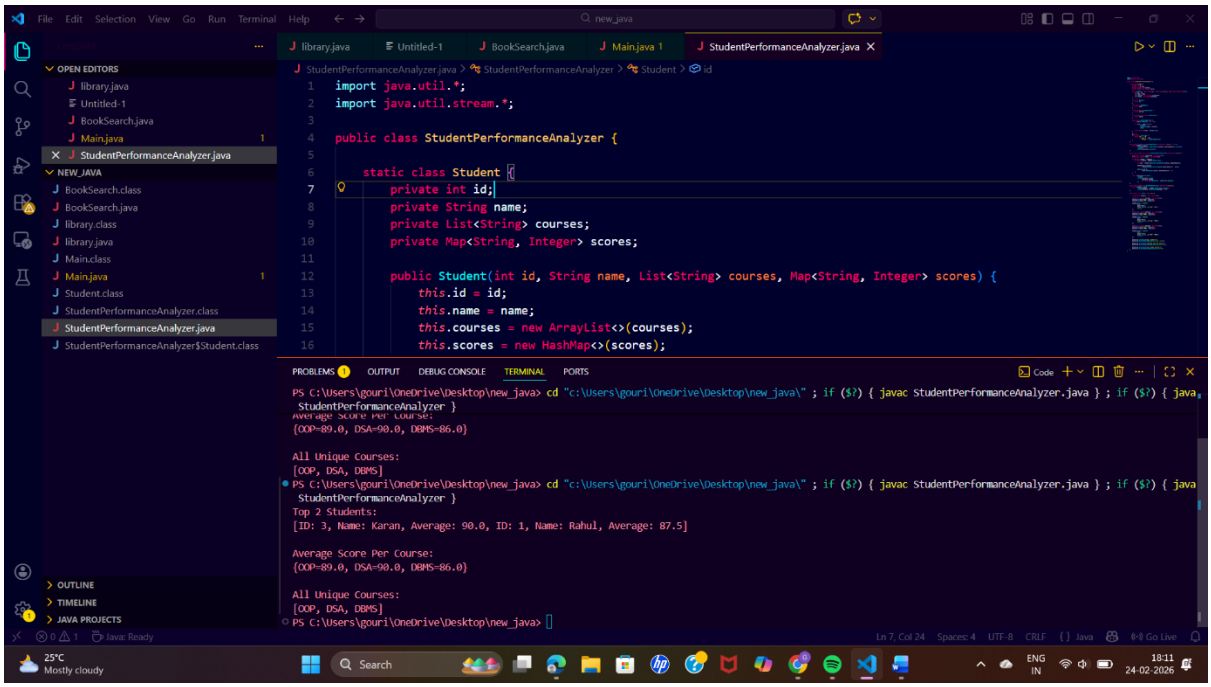
Answer:

```
import java.util.*;
import java.util.stream.*;

public class StudentPerformanceAnalyzer {

    static class Student {
        private int id;
        private String name;
        private List<String> courses;
        private Map<String, Integer> scores;

        public Student(int id, String name, List<String> courses,
Map<String, Integer> scores) {
            this.id = id;
            this.name = name;
            this.courses = new ArrayList<>(courses);
            this.scores = new HashMap<>(scores);
        }

        public int getId() {
            return id;
        }

        public String getName() {
            return name;
        }

        public List<String> getCourses() {
            return courses;
        }

        public Map<String, Integer> getScores() {
            return scores;
        }
```

```
        public double getAverageScore() {
            if (scores.isEmpty()) return 0.0;

            int total = scores.values()
                    .stream()
                    .mapToInt(Integer::intValue)
                    .sum();

            return (double) total / scores.size();
        }

        @Override
        public String toString() {
            return "ID: " + id +
                    ", Name: " + name +
                    ", Average: " + getAverageScore();
        }
    }

    public static List<Student> getTopNStudents(List<Student>
students, int n) {
        return students.stream()
                .sorted(Comparator.comparingDouble(Student::getAvera
geScore).reversed())
                .limit(n)
                .collect(Collectors.toList());
    }

    public static Map<String, Double>
getAverageScorePerCourse(List<Student> students) {

        Map<String, Integer> totalScores = new HashMap<>();
        Map<String, Integer> count = new HashMap<>();

        for (Student student : students) {
            for (String course : student.getCourses()) {

                int score = student.getScores().getOrDefault(course,
0);

                totalScores.put(course,
                        totalScores.getOrDefault(course, 0) +
score);

                count.put(course,
                        count.getOrDefault(course, 0) + 1);
            }
        }

        return totalScores.entrySet()
                .stream()
                .collect(Collectors.toMap(
                        Map.Entry::getKey,
                        e -> (double) e.getValue() /
count.get(e.getKey())
                ));
```

```
    }

    public static Set<String> getAllUniqueCourses(List<Student>
students) {
        return students.stream()
                .flatMap(student -> student.getCourses().stream())
                .collect(Collectors.toCollection(HashSet::new));
    }

    public static void main(String[] args) {

        List<Student> students = new ArrayList<>();

        Map<String, Integer> s1Scores = new HashMap<>();
        s1Scores.put("DSA", 85);
        s1Scores.put("OOP", 90);

        students.add(new Student(
                1,
                "Rahul",
                Arrays.asList("DSA", "OOP"),
                s1Scores
        ));

        Map<String, Integer> s2Scores = new HashMap<>();
        s2Scores.put("DSA", 95);
        s2Scores.put("DBMS", 80);

        students.add(new Student(
                2,
                "Anita",
                Arrays.asList("DSA", "DBMS"),
                s2Scores
        ));

        Map<String, Integer> s3Scores = new HashMap<>();
        s3Scores.put("OOP", 88);
        s3Scores.put("DBMS", 92);

        students.add(new Student(
                3,
                "Karan",
                Arrays.asList("OOP", "DBMS"),
                s3Scores
        ));

        System.out.println("Top 2 Students:");
        System.out.println(getTopNStudents(students, 2));

        System.out.println("\nAverage Score Per Course:");
        System.out.println(getAverageScorePerCourse(students));

        System.out.println("\nAll Unique Courses:");
        System.out.println(getAllUniqueCourses(students));
    }
}
```

Output of the code:



Complexity analysis:

Let:

- N = number of students
- M = average number of courses per student

---

1. Time Complexity of Computing Course Averages

To compute the average score per course, the algorithm iterates through:

- Each student → N
- Each course taken by each student → M

For every student–course pair, constant-time operations are performed using HashMap methods like getOrDefault() and put(), which run in O(1) on average.

Therefore, the total time complexity is:

$$O(N \times M)$$

After collecting totals, we iterate over all unique courses to compute the final averages.

In the worst case, the number of unique courses is at most M, so this does not change the overall complexity.

Final Time Complexity:

$$O(N \times M)$$

2. Time Complexity of Sorting Top N Students

To get the top N students, the program:

1. Computes the average score for each student → O(N)
2. Sorts all students based on average score using Comparator

Sorting N students requires:

$$O(N\log\ N)$$

The limit(n) operation after sorting runs in constant time relative to sorting.

Final Time Complexity:

$$O(N\log\ N)$$